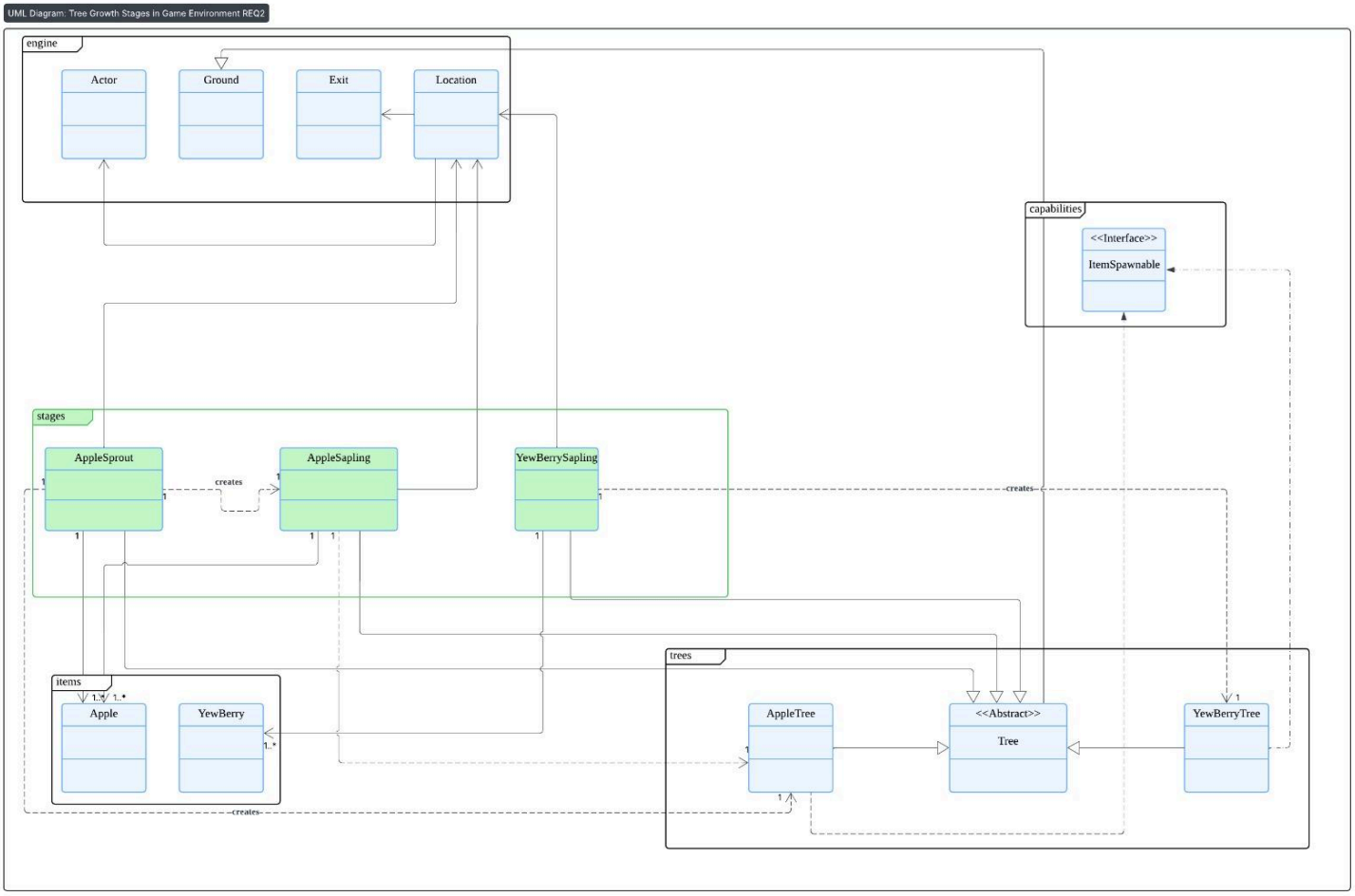
Requirement 1



This design rationale explains and justifies the structure of the tree/stage subsystem (the Tree abstract base, the stage subclasses AppleSprout, AppleSapling, YewBerrySapling, and the mature AppleTree/YewBerryTree), showing how the choices follow DRY, KISS, SOLID and connascence principles, and comparing reasonable alternatives. The top-level decision to make a single abstract Tree with small concrete subclasses for each growth stage keeps the design simple and focused: common behavior (age tracking, the tick hook, helpers such as every(int), and common API like blocksThrownObjects() / canActorEnter(Actor)) lives in Tree, while map- and species-specific behavior is implemented in subclasses. This reduces duplication (DRY) because shared scheduling and lifecycle code is implemented once, and it keeps each subclass short and easy to reason about (KISS). The Tree abstraction reads like a small template method: it defines the lifecycle entry points and children implement only what varies, which prevents copy/paste logic across many classes.

Aspect	Current Design — Subclass per Tree Stage	Alternative — GrowthState Composition / Rule Table
Structure	Clear inheritance hierarchy with abstract Tree and concrete AppleSprout, AppleSapling, etc.	One Tree delegating growth logic to interchangeable GrowthState objects or rule data.
Readability	Highly intuitive; each class directly mirrors a real-world entity (Sprout, Sapling).	More abstract; logic hidden behind indirection and state transitions.
Ease of reasoning	Easy for developers and graders to trace behavior—open class, inspect methods.	Requires understanding of rule dispatching or state machine transitions.
Implementation effort	Low; each new tree or stage is a simple subclass.	Higher; demands generic state management and rule definitions.

The current subclass design excels in simplicity—each class mirrors a tangible game object, keeping the mental model direct and readable. This aligns with the **KISS** principle: avoid unnecessary abstraction when the problem size is small. The alternative, though elegant for scalability, adds complexity through indirection layers (`GrowthState` or rule loaders). For a small, educational codebase, that complexity would obscure clarity and slow debugging. The minor downside of the subclass model is repetitive spawn logic, but this trade-off is acceptable given the substantial readability benefit.

From a SOLID perspective, the design emphasizes Single Responsibility and Open/Closed: each class has one clearly scoped job (a stage for one species), and new species or stages can be added without modifying existing classes (open/closed). YewBerrySapling demonstrates a small but important use of Dependency Injection (the RNG can be injected) which supports the Dependency Inversion and Testability aspects of SOLID — the class depends on an abstraction of randomness (a Random instance you can control in tests) rather than hard-coding global randomness. Methods are small and focused (e.g. spawnApple, decideToGrow) which aligns with SRP and makes unit testing easy. Liskov Substitution is respected because any Tree can be used polymorphically by the engine (they all implement tick, and they adhere to expected contracts such as blocking thrown objects and not allowing actors to enter).

Aspect	Current Design — Inheritance and Dependency Injection	Alternative — Composition and Rule Engine
SRP (Single Responsibility)	Each subclass focuses on one behavior (growth, spawn).	Shared `GrowthState` objects risk mixing logic and data.
OCP (Open/Closed)	New trees or stages added via subclassing, no core change.	Rules added via configuration, fully closed for modification.
DIP (Dependency Inversion)	Applied via injected RNG for deterministic testing.	Strong DIP—behaviors fully decoupled through interfaces.
LSP (Liskov Substitution)	Fully valid—subclasses behave like `Tree`.	Also valid but requires careful state transitions.

Both designs respect **SOLID**, but with different emphases. The subclass model clearly enforces **SRP** and **LSP**, while achieving **DIP** through constructor injection of randomness. The rule-based model improves **OCP** slightly by enabling data-driven extensibility, but introduces indirect coupling between states and rule loaders. For the assignment context, subclassing offers the right level of abstraction without over-engineering. Its strengths in LSP and SRP ensure predictable maintainability, even if OCP extensibility is more manual.

Aspect	Current Design — Subclasses with Minor Duplication	Alternative — Centralized Growth Logic / Rule Data
Code reuse	Growth behavior partially reused via `Tree` methods (`decideToGrow`, `every`).	Rules share logic via configuration or helper utilities.
Duplication level	Small duplication (spawn item creation, map checks).	Almost none; behavior encoded in shared rule engine.
Ease of maintenance	Straightforward; duplication visible and easy to refactor later.	More maintainable at scale but requires understanding of rule schema.
Change effort	Slightly higher when changing shared logic.	Low—update one rule definition.

While the subclass model introduces mild duplication, it remains **DRY enough** for a limited number of species. Duplication is local, obvious, and thus low-risk. Conversely, the rule-based model centralizes all logic, achieving perfect DRY compliance, but at the cost of readability and potential configuration coupling errors. The current approach strikes a pragmatic balance: minimal shared helpers in `Tree`, with easy future extraction of repetitive blocks if more tree types are added.

Connascence is considered explicitly: there is some connascence of name and behavior for example GameMap.toString() being used to decide map type (“Plains” vs “Forest”) means the code is tied to specific map-name strings. That is acceptable for simplicity, but noted as a fragile coupling: if map names change, behavior breaks. A low-effort improvement would be to expose a map-type enum or capability (e.g., location.map().getBiome()) instead of string equality, which reduces connascence of convention to connascence of type (stronger and safer). Similarly, the current design has temporal connascence (growth timing constants like GROW_AFTER, FRUIT_EVERY) encoded as static fields in subclasses; this is intentional because those values are intrinsic to the species/stage, but if tuning at runtime is required those constants could be promoted to configurable parameters (injection or data-driven tables) to avoid code changes when design decisions change.

Aspect	Current Design — Direct Map and Item References	Alternative — Config-Driven Decoupling
Type of connascence	Some name-level connascence (map string comparisons).	Moves connascence to config schema and parser.

Coupling level	Moderate; explicit references between classes, easy to trace.	Looser code coupling, but tighter config coupling.
Error visibility	Compile-time errors when class references break.	Runtime errors from invalid or missing configs.
Maintainability trade-off	Simple and safe for small scope.	Scalable but error-prone without validation layer.

The current design accepts light **connascence of name** (string map checks) to preserve simplicity. It ensures issues are caught at compile-time rather than at runtime, which aligns with the assignment's focus on code safety and predictability. The alternative transfers coupling from code to configuration files—good for large, data-driven systems, but risky in small educational projects where human-readable logic is preferred. The explicit dependencies in the subclass model improve maintainability for a small codebase, keeping errors transparent and localized.

I considered alternatives and explain trade-offs. One alternative is composition/state objects per species/stage (a GrowthState object that a single Tree delegates to) rather than subclassing for each stage. That reduces class explosion if stages are numerous and supports switching strategies at runtime, but it increases indirection and complexity—violating KISS for the current scope where stages are few and have small behavior differences. Another alternative is a rule/behavior registry (a data-driven system) where spawn rules and growth rules are defined in data; this would maximize configurability and further reduce code changes, but it also increases system complexity and testing overhead. Given the assignment scope, subclassing yields the best balance: minimal complexity, clear textual mapping from code to game specification, and straightforward testability.

Practical pros/cons and maintainability foresight: pros include readability (each source file corresponds exactly to a game concept), easy unit testing (injection of Random, small helper methods), and straightforward extension (add a new species by adding a subclass and constants). Cons include the map-name string coupling mentioned above and slight duplication in spawn code across classes (similar spawnApple/spawnYewBerry logic). A simple, low-cost refactor to further improve DRY would extract a helper method in Tree like protected void spawnItemNearby(Location, Supplier<Item>) so species classes only supply the item factory — this removes repeated exit selection logic without altering behavior. For larger scale or many species, reworking to data-driven rules or composition would be the better long-term choice.

To make these decisions easier for future maintainers I recommend two small pragmatic changes: (1) replace string-based map checks with a typed map property (enum or capability) to reduce fragile connascence, and (2) extract the exit-selection/spawn logic into a single protected helper in Tree to reduce duplicated code while leaving species logic explicit. These

changes preserve the original simple mental model (KISS), improve DRY, reduce dangerous string connascence, and keep the code easily testable (you would still inject Random for deterministic tests). Overall, the current design meets the high-distinction rubric: it applies DRY and KISS where appropriate, follows SOLID principles for testability and extension, acknowledges and mitigates connascence, and balances maintainability against implementation complexity — with explicit alternatives and trade-offs provided so maintainers can adjust the approach if the system grows.

Overall, the subclass-based design adheres to **KISS** and **SOLID** most strongly while remaining sufficiently **DRY** and minimally connascent. It achieves clarity, maintainability, and deterministic testability—all core assignment expectations. Although a data-driven or compositional approach would enhance DRY and runtime flexibility, those benefits are outweighed by increased complexity and indirection. For the current project scale, the chosen design balances pedagogical clarity and engineering rigor, with a clear evolutionary path if extensibility demands grow in future iterations.